

Serviços *Web* ou *Web Services*

STD29006 – Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Sumário

- 1 Definições gerais
- 2 *Web Services* SOAP
- 3 REST estilo de arquitetura de software

Definições gerais

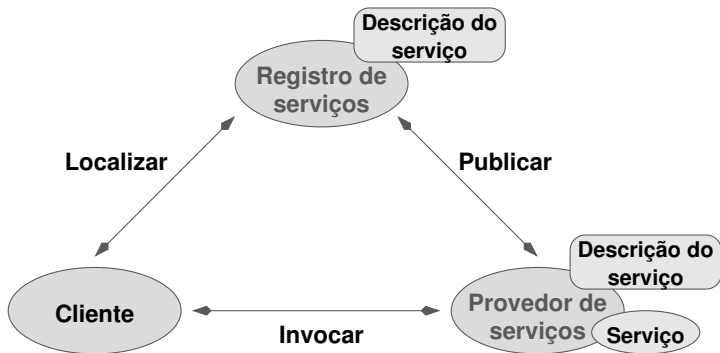
Arquitetura Orientada a Serviços

Service Oriented Architecture – SOA

Organiza aplicações como um conjunto de **serviços** que interagem por meio de **interfaces e protocolos padronizados**

- Interfaces são auto-descritivas e usam padrões abertos
- Três papéis principais:
 - **Diretório de serviços:** Repositório para publicar e localizar interfaces dos serviços
 - **Provedor de serviços:** Publica as interfaces e atende às requisições dos clientes
 - **Cliente:** Aplicação ou serviço que consome os serviços disponíveis

Arquitetura Orientada a Serviços



O que são serviços

- **Independente de tecnologia**

- Pode ser acessado por meio de protocolos e padrões amplamente utilizados.

- **Fraco acoplamento**

- Não exige conhecimento da implementação interna do cliente ou do serviço.

- **Localização transparente**

- Definição e localização do serviço são publicadas em repositórios acessíveis a diferentes clientes, independentemente de onde estejam.

Serviços Web (*Web Services*)

- Serviço identificado por uma **URL** e baseado em **padrões abertos** amplamente utilizados, como HTTP e XML
- **Auto-descritivos**: cliente e provedor se preocupam apenas com o formato e conteúdo das mensagens, sem depender de detalhes internos de implementação (*fraco acoplamento*)
- Oferecem uma **coleção de operações** sobre recursos (programas, objetos, bancos de dados), acessíveis por clientes via Internet
- Facilitam a comunicação através de firewalls, pois normalmente utilizam as portas 80 e 443, que costumam estar liberadas

Web Service vs. Web Server

Atenção

- **Web server:** software que recebe e responde a requisições HTTP (ex: Apache, Nginx, Caddy)
- **Web service:** aplicação que expõe operações e dados via interface padronizada, acessível por outros sistemas

Representação dos dados

Representação externa de dados

Forma padronizada de representar dados para comunicação entre sistemas diferentes.

■ Representação binária

- XDR (*eXternal Data Representation*)
- Serialização de objetos Java
- Google Protocol Buffers

■ Representação textual

- XML (*eXtensible Markup Language*)
- JSON (*JavaScript Object Notation*)

eXtensible Markup Language (XML)

- Usuários podem criar suas próprias marcações (*tags*)
- A ordem das *tags* importa (no HTML não importa)

```
<pessoa>  
  <nome>Alice</nome>  
  <sobrenome>dos Santos</sobrenome>  
  <idade>20</idade>  
  <telefones>  
    <numero tipo="residencial">48 3381 2800</numero>  
    <numero tipo="celular">48 93381 2800</numero>  
  </telefones>  
</pessoa>
```

JavaScript Object Notation (JSON)

Formato compacto, de padrão aberto independente, de troca de dados – <http://json.org>

- Um **objeto** é um conjunto não ordenado de pares chave:valor, encapsulados por chaves { e } e separados por vírgula

```
{  
  "nome": "Alice",  
  "sobrenome": "dos Santos",  
  "idade": 20,  
  "telefones": [  
    {  
      "tipo": "residencial",  
      "numero": "48 3381 2800"  
    },  
    {  
      "tipo": "celular",  
      "numero": "48 93381 2800"  
    }  
  ]  
}
```

- Strings são delimitadas por aspas
- Números são representados obrigatoriamente na forma decimal e seguem a sintaxe de representação do C, C++, Java
- Vetor é uma coleção ordenada de valores, delimitados por colchetes
 - Pode conter string, número, true, false, null, objeto ou vetor

Representação de um menu XML x JSON (fonte: Wikipedia)

```
<menu id="file" value="File">
  <popup>
    <menuitem value="New" onclick="CreateNewDoc()" />
    <menuitem value="Open" onclick="OpenDoc()" />
    <menuitem value="Close" onclick="CloseDoc()" />
  </popup>
</menu>
```

```
{
  "menu": {
    "id": "file",
    "value": "File",
    "popup": {
      "menuitem": [
        { "value": "New", "onclick": "CreateNewDoc()" },
        { "value": "Open", "onclick": "OpenDoc()" },
        { "value": "Close", "onclick": "CloseDoc()" }
      ]
    }
  }
}
```

Arquitetura dos Serviços Web

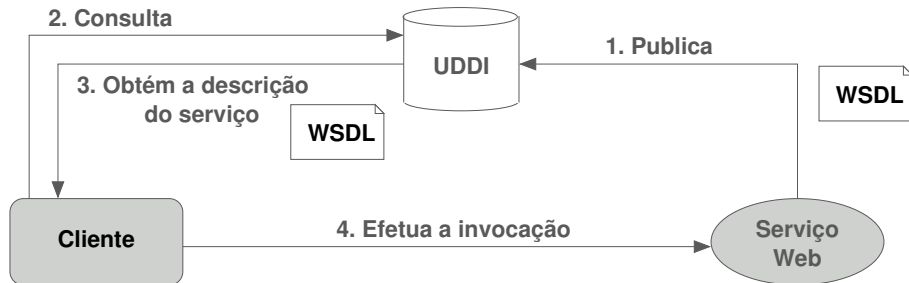
- Existem duas principais abordagens para implementar um serviço web:
 - **Baseada no protocolo SOAP**
 - Utiliza o protocolo SOAP para troca de mensagens em formato XML
 - Possui várias especificações complementares para segurança, orquestração, etc. (ex: WS-Policy, WS-Security, WS-Trust)
 - **Baseada no estilo REST**
 - Utiliza os princípios do *Representational State Transfer* (REST).
 - Fundamenta-se nos padrões da web: HTTP, URI, HTML, XML e JSON
- A escolha entre SOAP e REST depende das necessidades da aplicação

Web Services SOAP

Arquitetura dos Serviços Web: abordagem com SOAP

- 1 **Descrever a interface do serviço** usando uma linguagem padrão
 - Exemplo: *Web Services Description Language* (WSDL)
- 2 **Publicar a interface** em um documento XML acessível
 - Exemplo: *Universal Description, Discovery and Integration* (UDDI)
- 3 **Trocar mensagens XML** entre cliente e serviço
 - Utilizando o protocolo SOAP

Arquitetura dos Serviços Web



Descrição dos serviços

A descrição com WSDL responde:

- Quais serviços estão disponíveis?
- Como acessar cada serviço?
- Quais dados são necessários para usar um serviço?
- Como cliente e servidor entendem as requisições?
- Em qual formato as informações são enviadas ao cliente?

Web Services Description Language – WSDL

Gramática em XML para especificar interfaces de Serviços *Web*

- Operações disponíveis
- Tipos de dados
- Protocolos utilizados
- Localização do serviço

WSDL é composto por três elementos principais

■ types

- especifica a estrutura e o conteúdo das mensagens

■ messages e operation

- descreve as operações que poderão ser executadas

■ port

- especifica como mapear as operações nos protocolos de rede utilizados para o transporte

Documento
WSDL



Tipos dos dados das mensagens

Operações abstratas

Associação dos Serviços

Transporte

Exemplo WSDL: Calculadora.java

```
public class Calculadora{

    public int soma(int i1, int i2){
        return i1 + i2;
    }

    public int subtracao(int i1, int i2){
        return i1 - i2;
    }
}
```

Documento WSDL

```
<wsdl:definitions targetNamespace="http://localhost/Calculadora.jws">

  <wsdl:message name="subtracaoRequest">
    <wsdl:part name="i1" type="xsd:int"/>
    <wsdl:part name="i2" type="xsd:int"/>
  </wsdl:message>

  <wsdl:message name="somaRequest">
    <wsdl:part name="i1" type="xsd:int"/>
    <wsdl:part name="i2" type="xsd:int"/>
  </wsdl:message>

  <wsdl:message name="subtracaoResponse">
    <wsdl:part name="subtracaoReturn" type="xsd:int"/>
  </wsdl:message>

  <wsdl:message name="somaResponse">
    <wsdl:part name="somaReturn" type="xsd:int"/>
  </wsdl:message>

</wsdl:definitions>
```

Documento WSDL

```
<wsdl:portType name="Calculadora">

  <wsdl:operation name="soma" parameterOrder="i1 i2">
    <wsdl:input message="impl:somaRequest" name="somaRequest"/>
    <wsdl:output message="impl:somaResponse" name="somaResponse"/>
  </wsdl:operation>

  <wsdl:operation name="subtracao" parameterOrder="i1 i2">
    <wsdl:input message="impl:subtracaoRequest" name="subtracaoRequest"/>
    <wsdl:output message="impl:subtracaoResponse" name="subtracaoResponse"/>
  </wsdl:operation>

</wsdl:portType>
```

Documento WSDL

```
<wsdl:service name="CalculadoraService">  
  <wsdl:port binding="impl:CalculadoraSoapBinding" name="Calculadora">  
    <wsdlsoap:address location="http://localhost/Calculadora.jws"/>  
  </wsdl:port>  
</wsdl:service>  
</wsdl:definitions>
```


SOAP - Protocolo para troca de mensagens (RPC)

■ Envelope

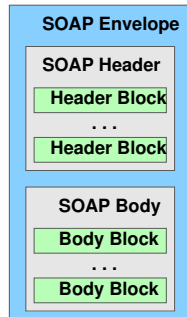
- Elemento raiz no documento XML que representa a mensagem SOAP

■ Header (opcional)

- Oferece mecanismos para estender a mensagem
- Informações de roteamento, segurança, coordenação

■ Body

- Document - A RPC em si faz parte deste elemento
- Fault (opcional)



Comunicação pode ser síncrona ou assíncrona. Mensagens SOAP podem ser transportadas sobre HTTP, SMTP, JMS, ...

Pedido SOAP

Invocação do método `soma` e passando 2 parâmetros

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <soma soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <op1 xsi:type="xsd:int">10</op1>
      <op2 xsi:type="xsd:int">8</op2>
    </soma>
  </soapenv:Body>
</soapenv:Envelope>
```

Resposta SOAP

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <somaResponse soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <somaReturn xsi:type="xsd:int">
        18
      </somaReturn>
    </somaResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

REST estilo de arquitetura de software

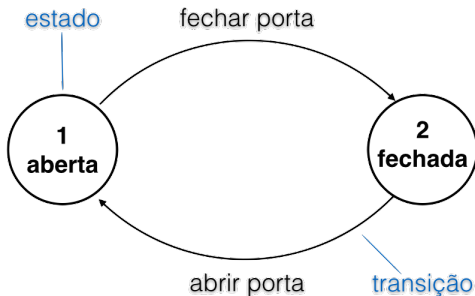
Representational State Transfer (REST) I

- REST é um estilo arquitetural para construção de sistemas distribuídos, especialmente aplicações web.
- Foi definido por Roy Fielding em sua tese de doutorado em 2000.
- Baseia-se nos princípios fundamentais da web, como recursos identificados por URIs, comunicação sem estado e uso dos métodos HTTP.
- REST facilita a escalabilidade, simplicidade e interoperabilidade entre sistemas.

REpresentational State Transfer (REST) II

REST e o comportamento das aplicações web

Recursos são organizados como páginas conectadas por *links*. Cada link representa uma transição de estado, levando o usuário de uma página (estado) para outra. Assim, a navegação pelos links altera o estado da aplicação.



Fundamentos do REST

- 1 **Recursos:** Tudo que pode ser acessado na web é um recurso, identificado por uma URI única.
- 2 **Métodos HTTP:** As operações sobre os recursos são feitas usando os verbos HTTP: GET (buscar), POST (criar), PUT (atualizar) e DELETE (remover).
- 3 **Sem estado (stateless):** Cada requisição traz todas as informações necessárias; o servidor não armazena contexto entre as requisições.

SOAP apenas usa HTTP para transportar mensagens. REST utiliza HTTP como parte fundamental da lógica da aplicação.

Fundamentos do REST

Recursos

- Qualquer informação pode ser caracterizada como um recurso
 - Ex: página, imagem, coleção de recursos, etc
- Recursos são **identificados por uma URI**
 - Identificador único e que permite endereçar um recurso na web
 - Uma URI identifica um único recurso, porém um recurso pode possuir mais de uma URI (relação muitos-para-um)

Formato da URI

```
scheme://host:port/path?queryString#fragment
```


Estrutura das URIs em REST

Identificação de Recursos

- As URIs em REST seguem uma estrutura hierárquica, permitindo acessar recursos de forma organizada e intuitiva
- Cada segmento do caminho representa um recurso mais específico dentro do anterior
- Exemplo:
 - `/campus` – coleção de campi
 - `/campus/sj` – campus específico (São José)
 - `/campus/sj/blocos/a` – bloco A do campus SJ
 - `/campus/sj/blocos/a/laboratorios/sidi` – laboratório SIDI no bloco A do campus SJ
- Parâmetros de consulta (*query string*) podem ser usados para filtrar ou detalhar ainda mais a busca, como em `?computador=1`

Ações – Verbos HTTP

Como clientes e servidores trocam representações de recursos

- **GET**
(Buscar dados)
- **POST**
(Criar novo recurso)
- **PUT**
(Atualizar recurso existente)
- **DELETE**
(Remover recurso)

Ações – Verbos HTTP

Como clientes e servidores trocam representações de recursos

■ **GET**
(Buscar dados)

■ **POST**
(Criar novo recurso)

■ **PUT**
(Atualizar recurso existente)

■ **DELETE**
(Remover recurso)

■ Obter lista de todos os campi:

```
GET http://localhost/campus
```

■ Obter capacidade do laboratório de Sistemas Digitais:

```
GET http://localhost/campus/laboratorios/sidi/  
capacidade
```

Ações – Verbos HTTP

Como clientes e servidores trocam representações de recursos

■ GET

(Buscar dados)

■ POST

(Criar novo recurso)

■ PUT

(Atualizar recurso existente)

■ DELETE

(Remover recurso)

■ Adicionar novo laboratório:

```
POST http://localhost/campus
Content-Type: application/json

{ "bloco": "...", "laboratorio": "...", "equipamentos":
  [...], "capacidade": ... }
```

Ações – Verbos HTTP

Como clientes e servidores trocam representações de recursos

■ GET

(Buscar dados)

■ POST

(Criar novo recurso)

■ PUT

(Atualizar recurso existente)

■ DELETE

(Remover recurso)

■ Atualizar informações do laboratório de Sistemas Digitais:

```
PUT http://localhost/campus/laboratorios/sidi
```

```
Content-Type: application/json
```

```
{ "bloco": "...", "laboratorio": "...", "equipamentos":  
  [...], "capacidade": ... }
```

Ações – Verbos HTTP

Como clientes e servidores trocam representações de recursos

- **GET**

(Buscar dados)

- **POST**

(Criar novo recurso)

- **PUT**

(Atualizar recurso existente)

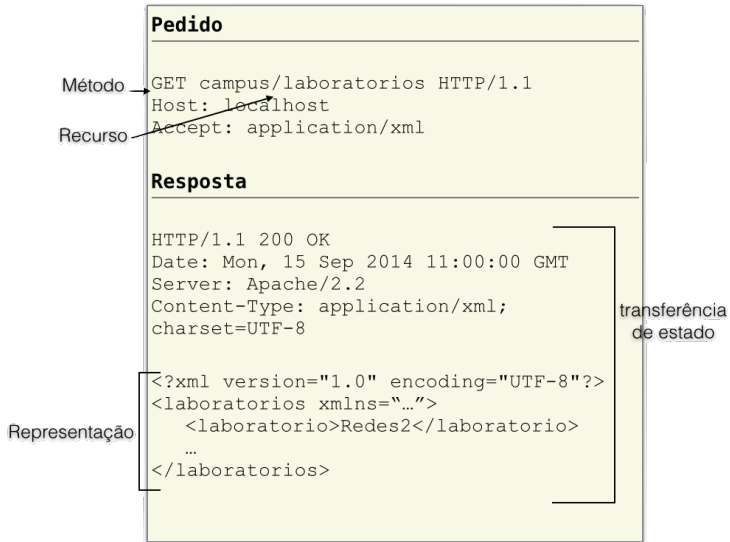
- **DELETE**

(Remover recurso)

- Remover o laboratório de Redes:

```
DELETE http://localhost/campus/laboratorios/redes
```

Pedido e resposta para um Serviço *Web* REST



REST vs SOAP

- Chamada de Procedimento Remoto (RPC), como é o caso do SOAP, tem foco na diversidade de operações

```
obterCapacidade(int id);  
  
definirCapacidade(int id, int capacidade);
```

- REST tem foco na diversidade de recursos

```
GET http://localhost/laboratorios/sidi/capacidade  
  
PUT http://localhost/laboratorios
```


Comparando SOAP e REST na obtenção de dados

Exemplo de requisição para obter a capacidade de um laboratório

- Pedido gerado por um cliente para *Web Service* SOAP

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" >
  <soapenv:body lab="http://localhost/campus/laboratorio">
    <lab:getCapacidade>
      <lab:id>SiDi</lab:id>
    </lab:getCapacidade>
  </soapenv:body>
</soapenv:Envelope>
```

- Pedido gerado por um cliente para *Web Service* REST

```
GET http://localhost/campus/laboratorios/SiDi/capacidade
```

Restrições arquiteturais do REST

■ Arquitetura cliente-servidor

- Sem estado (*Stateless*)
- Suporte a *cache*
- Organização em camadas
- Código sob demanda (opcional)
- Interface uniforme
- Separação clara entre cliente (quem consome) e servidor (quem fornece o recurso)
- Comunicação feita por meio do protocolo HTTP

Restrições arquiteturais do REST

- Arquitetura cliente-servidor
- Sem estado (*Stateless*)
- Suporte a *cache*
- Organização em camadas
- Código sob demanda (opcional)
- Interface uniforme
- Cada requisição é independente: o servidor não armazena informações do cliente entre diferentes pedidos

Restrições arquiteturais do REST

- Arquitetura cliente-servidor
- Sem estado (*Stateless*)
- Suporte a *cache*
- Organização em camadas
- Código sob demanda (opcional)
- Interface uniforme
- Respostas podem ser armazenadas em *cache* por clientes ou intermediários (como CDNs), reduzindo a carga do servidor e melhorando o desempenho

Restrições arquiteturais do REST

- Arquitetura cliente-servidor
- Sem estado (*Stateless*)
- Suporte a *cache*
- Organização em camadas
- Código sob demanda (opcional)
- Interface uniforme
- O sistema pode ser dividido em várias camadas (por exemplo, proxies, balanceadores de carga), sem que o cliente precise saber disso

Restrições arquiteturais do REST

- Arquitetura cliente-servidor
- Sem estado (*Stateless*)
- Suporte a *cache*
- Organização em camadas
- Código sob demanda (opcional)
- Interface uniforme
- O servidor pode enviar código para ser executado no cliente (por exemplo, JavaScript), tornando o comportamento mais flexível

Restrições arquiteturais do REST

- Arquitetura cliente-servidor
- Sem estado (*Stateless*)
- Suporte a *cache*
- Organização em camadas
- Código sob demanda (opcional)
- Interface uniforme
- Recursos são identificados por URIs
- Manipulação dos recursos por meio de suas representações (por exemplo, apagar, atualizar)
- Mensagens são auto-descritivas, informando como devem ser processadas (por exemplo, via *MediaType*)
- Uso de hipermídia para navegação entre estados da aplicação (HATEOAS)

API RESTful vs API REST

Interface de programação de aplicações (API)

Conjunto de funções, métodos ou classes que permitem a integração entre sistemas

- Uma **API RESTful** segue todas as restrições do REST
- APIs que usam apenas parte dos princípios REST são chamadas de APIs REST, mas não são RESTful

Projeto de API de *Web Services* RESTful

Versão na URI

- Sempre inclua a versão da API na URI (ex: v1, v2). Isso permite atualizar a API sem impactar aplicações que usam versões anteriores.

```
https://dominio/nome-da-api/v1/recurso
```

```
https://example.com/api/v1/campus
```

Projeto de API de *Web Services* RESTful

Identificação de recursos por URI

- Use substantivos para nomear recursos, evite verbos e preposições
- Utilize letras minúsculas, sem acentos, e o padrão kebab-case para URIs
- Use o plural para coleções (/cursos) e singular para itens individuais (/curso)
- Parâmetros na *query string* devem ser em camelCase OU snake_case

Como fazer

`https://example.com/api/v1/cursos`

`https://example.com/api/v1/cursos/nivel-superior/{id_curso}`

Como não fazer

`https://example.com/api/v1/listarCursos`

`https://example.com/api/v1/obter_dados_do_curso/nivel-superior/{id_curso}`

Projeto de API de *Web Services* RESTful

Verbos HTTP para operações CRUD

- Use os verbos HTTP para definir a ação:
 - GET para obter, POST para criar, PUT para atualizar e DELETE para excluir
- GET, PUT e DELETE são idempotentes (repetir a requisição não muda o resultado)
- GET é seguro: não altera o estado do servidor
- Não inclua o nome da operação (como "apagar" ou "criar") na URI; a ação é definida pelo verbo HTTP

```
# Como fazer
GET https://example.com/api/v1/cursos/1234
DELETE https://example.com/api/v1/cursos/1234

# Como nao fazer
GET https://example.com/api/v1/apagar/curso/1234
```

Projeto de API de *Web Services* RESTful

Cabeçalhos HTTP para tipos de dados

- Use Content-Type para informar o formato dos dados enviados
- Use Accept para informar o formato desejado da resposta

```
# Enviando e recebendo dados em JSON
```

```
POST /livros HTTP/1.1
```

```
Host: localhost
```

```
Accept: application/json
```

```
Content-Type: application/json
```

```
Content-Length: 37
```

```
{"titulo": "O livro", "autor": "Joao"}
```

Projeto de API de *Web Services* RESTful

Uso dos códigos de estado do HTTP para semântica da resposta

HTTP status	Descrição
200 OK	Requisição bem sucedida
201 CREATED	Requisição resultou na criação de um novo recurso
400 BAD REQUEST	Pedido mal formado
401 UNAUTHORIZED	Pedido não autorizado
404 NOT FOUND	Recurso não encontrado
405 METHOD NOT ALLOWED	Método não permitido para o recurso

Projeto de API de *Web Services* RESTful

Exemplo de verbos sobre recursos

Recurso	GET	POST	PUT	DELETE
/campus	Obtém lista	Cria novo	405 HTTP	405 HTTP
/campus/sj	Dados do campus	405 HTTP	Atualiza	Exclui
/campus/sj/blocos	Obtém lista	Cria novo	405 HTTP	405 HTTP

REST, SOAP, RESTful e APIs

- **SOAP:** protocolo padronizado para troca de mensagens entre aplicações, geralmente usando XML
- **REST:** estilo arquitetural que define princípios para comunicação entre sistemas usando padrões da web (HTTP, URIs, etc.)
- **RESTful:** API que segue todas as restrições e princípios do REST
- **API:** interface que permite a integração entre sistemas; pode ser baseada em REST, SOAP ou outros padrões

Modelo de Maturidade de Richardson para APIs REST I

- O **Modelo de Maturidade de Richardson**^{1,2} classifica APIs de *Web Services* em quatro níveis, conforme a adoção dos princípios REST.
- Quanto mais alto o nível, maior a aderência às restrições da arquitetura REST e maior a maturidade da API.

Modelo de Maturidade de Richardson para APIs REST II

■ Nível 0

- Uma única URI e um único método (geralmente POST)
- Cenário típico de *Web Services* SOAP

■ Nível 1

- Diversas URIs, mas ainda utilizando um único método HTTP

■ Nível 2

- Diversas URIs e uso adequado dos métodos HTTP (GET, POST, PUT, DELETE)

■ Nível 3

- Uso de hipermídia (HATEOAS) para navegação entre recursos
- Cada recurso descreve suas habilidades e interconexões

¹ <https://www.crummy.com/writing/speaking/2008-QCon/act3.html>

² <https://martinfowler.com/articles/richardsonMaturityModel.html>

Níveis de Maturidade de Richardson I

Exemplos

- **Nível 0:** Uma única URL e um único método (geralmente POST)

```
POST /api HTTP/1.1
Host: exemplo.com
Content-Type: application/json

{"acao": "criarUsuario", "nome": "Juca"}
```

- **Nível 1:** Diversas URLs, mas ainda utilizando um único método HTTP

```
POST /usuarios HTTP/1.1
Host: exemplo.com
Content-Type: application/json

{"nome": "Juca"}
```

Níveis de Maturidade de Richardson II

Exemplos

■ **Nível 2:** Diversas URIs e uso adequado dos métodos HTTP

```
GET /usuarios/1 HTTP/1.1
```

```
PUT /usuarios/1 HTTP/1.1  
Host: exemplo.com  
Content-Type: application/json  
  
{"nome": "Juca"}
```

Níveis de Maturidade de Richardson III

Exemplos

- **Nível 3:** Uso de hipermídia (HATEOAS) para navegação entre recursos

```
{  
  "nome": "Juca",  
  "links": [  
    {"rel": "self", "href": "/usuarios/1"},  
    {"rel": "amigos", "href": "/usuarios/1/amigos"}  
  ]  
}
```